



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

ASIGNATURA: DESARROLLO DE APLICACIONES MÓVILES

Tarea 1.1 Creación de una aplicación móvil A

Arquitectura Modelo-Vista-Controlador en Flutter

El documento presenta la resolución de 5 ejercicios algorítmicos aplicados al desarrollo móvil, estructurando el código para separar la lógica de negocio de la interfaz visual mediante el patrón MVC, incluyendo diagramas Nassi-Shneiderman y pseudocódigos.

Autores: Alejandro Montufar, Diana Guerra, Paul Moreno, Leonel Tipán

Fecha: 10 de mayo de 2026

Índice

1. Marco Teórico	2
1.1. Flutter	2
1.2. Dart	2
1.3. Android Studio	2
1.4. Patrón Modelo-Vista-Controlador (MVC)	2
2. Desarrollo de los Ejercicios	3
2.1. Ejercicio 1: Caja de Supermercado	3
2.1.1. Pseudocódigo	3
2.1.2. Diagrama Nassi-Shneiderman	3
2.1.3. Código (Lógica Central del Modelo)	4
2.1.4. Ejecución	4
2.2. Ejercicio 2: Máquina de Vuelto	5
2.2.1. Pseudocódigo	5
2.2.2. Diagrama Nassi-Shneiderman	5
2.2.3. Código (Lógica Central del Modelo)	6
2.2.4. Ejecución	6
2.3. Ejercicio 3: Año Bisiesto	7
2.3.1. Pseudocódigo	7
2.3.2. Diagrama Nassi-Shneiderman	7
2.3.3. Código (Lógica Central del Modelo)	7
2.3.4. Ejecución	8
2.4. Ejercicio 4: Números Perfectos	9
2.4.1. Pseudocódigo	9
2.4.2. Diagrama Nassi-Shneiderman	9
2.4.3. Código (Lógica Central del Modelo)	10
2.4.4. Ejecución	10
2.5. Ejercicio 5: Control de Peso (Club)	11
2.5.1. Pseudocódigo	11
2.5.2. Diagrama Nassi-Shneiderman	11
2.5.3. Código (Lógica Central del Modelo)	12
2.5.4. Ejecución	12
3. Conclusiones	13
4. Referencias	13
5. Anexos	13
5.1. Repositorio de código fuente	13
5.1.1. Enlace principal	13

1. Marco Teórico

1.1. Flutter

Flutter es un framework de desarrollo multiplataforma creado por Google que permite construir interfaces nativas para Android, iOS, web y escritorio desde una sola base de código. Su paradigma principal es la programación declarativa con *widgets*, donde la interfaz se describe como un árbol de componentes reutilizables. Flutter utiliza un motor gráfico propio para renderizado, lo que reduce la dependencia de componentes nativos y mantiene consistencia visual entre plataformas.

1.2. Dart

Dart es el lenguaje de programación usado por Flutter. Combina tipado estático opcional, soporte para programación orientada a objetos y características modernas como *async/await* para la concurrencia. Su compilador permite generar código nativo de alto rendimiento para dispositivos móviles y código JavaScript para la web, lo que habilita el enfoque multiplataforma de Flutter.

1.3. Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial para Android y uno de los más usados para Flutter. Proporciona herramientas de edición, depuración, emuladores y administración de dependencias. En proyectos Flutter integra la ejecución en dispositivos, el análisis estático y el soporte para hot reload, lo que acelera el ciclo de desarrollo.

1.4. Patrón Modelo-Vista-Controlador (MVC)

El patrón MVC es una arquitectura de diseño de software que separa los datos de una aplicación, la interfaz de usuario y la lógica de control en tres componentes altamente desacoplados:

- **Modelo:** Maneja la lógica de negocio y las reglas de procesamiento de datos independientes de la interfaz. Representa el núcleo del comportamiento de la aplicación.
- **Vista:** Proporciona la representación gráfica y captura los eventos táctiles del usuario (en Flutter, esto se realiza mediante el árbol de *widgets* y *Scaffolds*).
- **Controlador:** Recibe las interacciones del usuario desde la vista, las transforma en peticiones o ejecuciones para el modelo, y devuelve los resultados limpios para actualizar el estado de la vista.

2. Desarrollo de los Ejercicios

2.1. Ejercicio 1: Caja de Supermercado

Problema: En un supermercado un cajero captura los precios de los artículos que los clientes compran e indica el monto a pagar. Al final del día indica a su supervisor cuánto cobró en total a todos los clientes.

2.1.1. Pseudocódigo

Inicio

totalClienteActual = 0

totalCajaDia = 0

Mientras haya artículos para el cliente actual Hacer

Leer precioArticulo

totalClienteActual = totalClienteActual + precioArticulo

FinMientras

montoAPagar = totalClienteActual

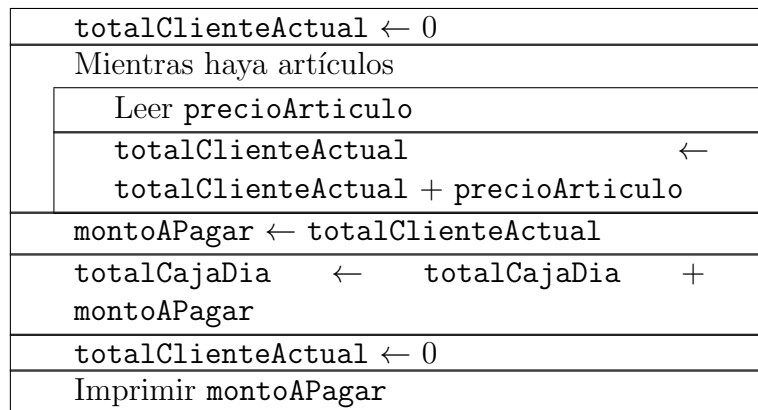
totalCajaDia = totalCajaDia + montoAPagar

totalClienteActual = 0

Imprimir montoAPagar

Fin

2.1.2. Diagrama Nassi-Shneiderman



2.1.3. Código (Lógica Central del Modelo)

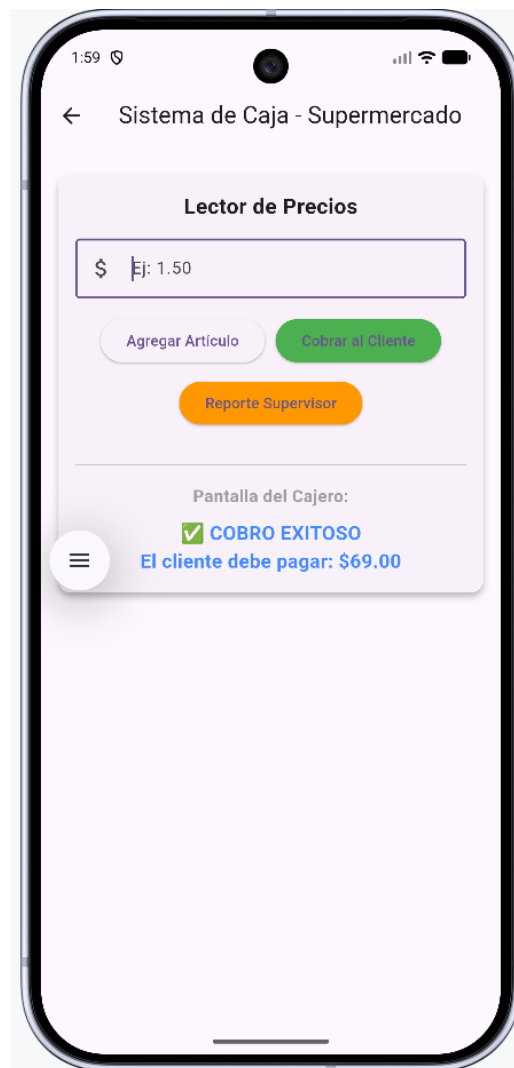
```
lib/model/caja_model.dart

class CajaModelo {
  double _totalClienteActual = 0.0;
  double _totalCajaDia = 0.0;
  int _clientesAtendidos = 0;

  void agregarArticulo(double precio) {
    _totalClienteActual += precio;
  }

  double cerrarCobroCliente() {
    double montoAPagar = _totalClienteActual;
    _totalCajaDia += montoAPagar;
    _clientesAtendidos++;
    _totalClienteActual = 0.0;
    return montoAPagar;
  }
}
```

2.1.4. Ejecución



2.2. Ejercicio 2: Máquina de Vuelto

Problema: Máquina que da vuelto con monedas de \$2, \$1, \$0.50, \$0.25 y \$0.10. Indicar el vuelto empleando el menor número posible de monedas.

2.2.1. Pseudocódigo

Inicio

Leer precio, pago

Si pago < precio Entonces

Imprimir "Pago insuficiente"

Sino

vuelto = pago - precio

centavos = vuelto * 100

m2 = centavos DIV 200; centavos = centavos MOD 200

m1 = centavos DIV 100; centavos = centavos MOD 100

m50 = centavos DIV 50; centavos = centavos MOD 50

m25 = centavos DIV 25; centavos = centavos MOD 25

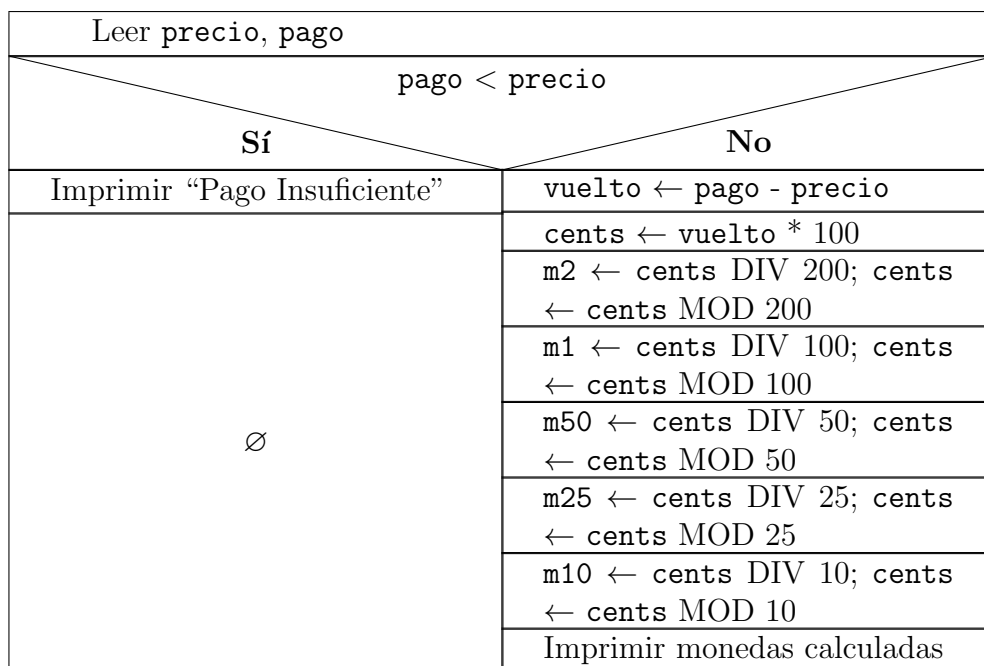
m10 = centavos DIV 10; centavos = centavos MOD 10

Imprimir m2, m1, m50, m25, m10

FinSi

Fin

2.2.2. Diagrama Nassi-Shneiderman



2.2.3. Código (Lógica Central del Modelo)

```
lib/model/vuelto_model.dart

static VueltoModelo calcular(double precio, double pago) {
  if (pago < precio) throw Exception("PagoInsuficiente");

  final vuelto = pago - precio;
  int centavos = (vuelto * 100).round();

  final m2 = centavos ~/ 200; centavos %= 200;
  final m1 = centavos ~/ 100; centavos %= 100;
  final m50 = centavos ~/ 50; centavos %= 50;
  final m25 = centavos ~/ 25; centavos %= 25;
  final m10 = centavos ~/ 10; centavos %= 10;

  return VueltoModelo(vueltoTotal: vuelto, monedas2: m2,
    monedas1: m1, monedas50: m50,
    monedas25: m25, monedas10: m10);
}
```

2.2.4. Ejecución



2.3. Ejercicio 3: Año Bisiesto

Problema: Un año es bisiesto si es múltiplo de 4, exceptuando los múltiplos de 100, que sólo son bisiestos cuando son múltiplos además de 400.

2.3.1. Pseudocódigo

Inicio

Leer anio

Si $(\text{anio} \bmod 4 == 0 \wedge \text{anio} \bmod 100 \neq 0) \vee (\text{anio} \bmod 400 == 0)$ Entonces
 esBisiesto = Verdadero

Sino

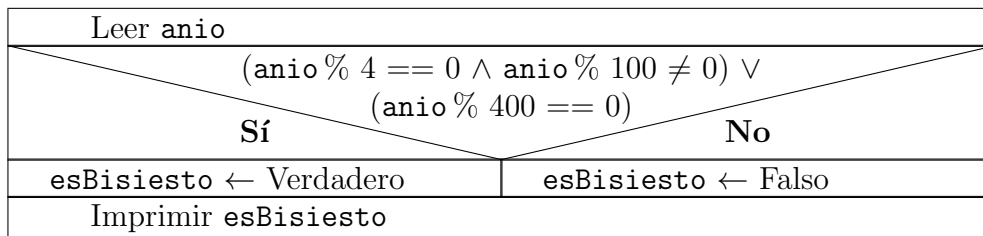
 esBisiesto = Falso

FinSi

Imprimir esBisiesto

Fin

2.3.2. Diagrama Nassi-Shneiderman



2.3.3. Código (Lógica Central del Modelo)

lib/model/bisiesto_model.dart

```
class BisiestoModel {  
  bool esBisiesto(int anio) {  
    return (anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0);  
  }  
}
```

2.3.4. Ejecución



2.4. Ejercicio 4: Números Perfectos

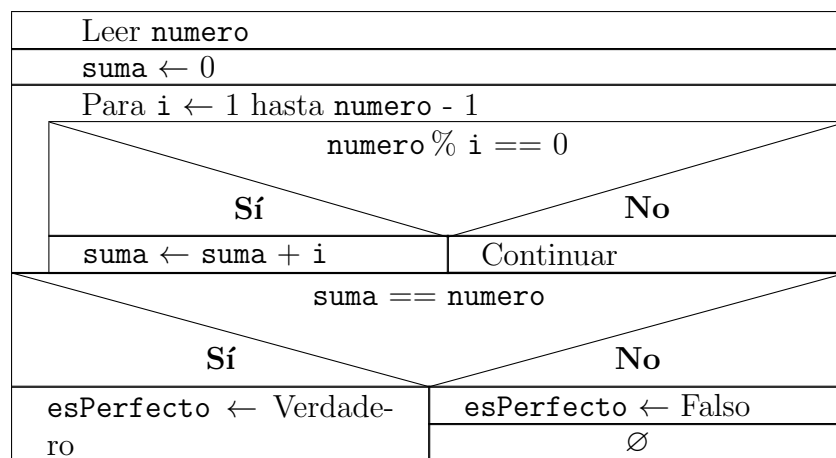
Problema: Un número N es perfecto si la suma de sus divisores (excluido el propio N) es N . Hacer un algoritmo que determine si un número es perfecto.

2.4.1. Pseudocódigo

```
Inicio
  Leer numero
  suma = 0
  Para i desde 1 hasta numero - 1 Hacer
    Si numero MOD i == 0 Entonces
      suma = suma + i
    FinSi
  FinPara

  Si suma == numero Entonces
    esPerfecto = Verdadero
  Sino
    esPerfecto = Falso
  FinSi
Fin
```

2.4.2. Diagrama Nassi-Shneiderman



2.4.3. Código (Lógica Central del Modelo)

```
lib/model/perfecto_model.dart

class PerfectoModel {
  bool esPerfecto(int numero) {
    if (numero <= 1) return false;

    int suma = 0;
    for (int i = 1; i < numero; i++) {
      if (numero % i == 0) {
        suma += i;
      }
    }
    return suma == numero;
  }
}
```

2.4.4. Ejecución



2.5. Ejercicio 5: Control de Peso (Club)

Problema: Se pesa a cada persona en diez básculas distintas para tener el promedio exacto. Si existe diferencia positiva entre el promedio y el último peso de la última reunión, subieron de peso; si es negativa, bajaron.

2.5.1. Pseudocódigo

Inicio

Leer pesos[10]

suma = 0

Para i desde 1 hasta 10 Hacer

 suma = suma + pesos[i]

FinPara

promedio = suma / 10

ultimoPeso = pesos[10]

diferencia = ultimoPeso - promedio

Si diferencia > 0 Entonces

 estado = "SUBIÓ"

Sino

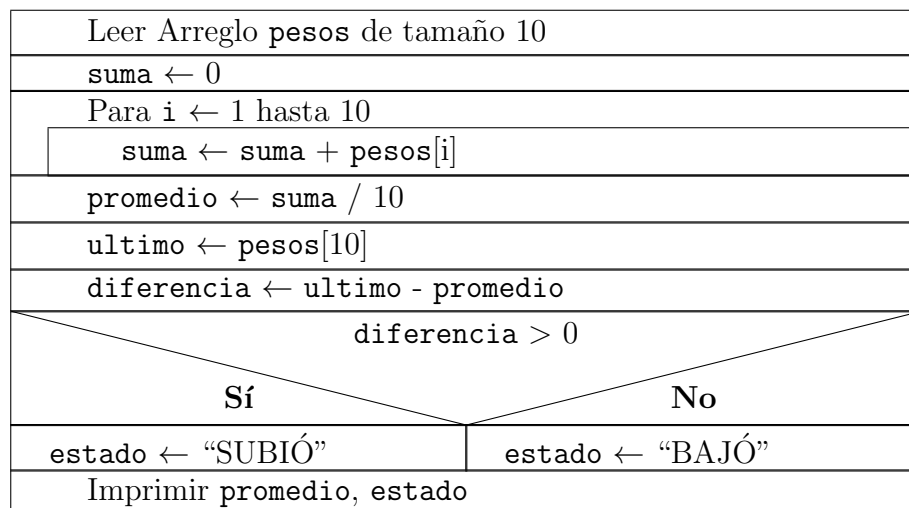
 estado = "BAJÓ"

FinSi

Imprimir promedio, estado, diferencia

Fin

2.5.2. Diagrama Nassi-Shneiderman



2.5.3. Código (Lógica Central del Modelo)

```
lib/model/persona_modelo.dart

static PersonaModelo analizar(List<double> pesos) {
  final suma = pesos.reduce((a, b) => a + b);
  final promedio = suma / pesos.length;
  final ultimo = pesos.last;

  final diferencia = ultimo - promedio;
  final estado = diferencia > 0 ? "SUBIÓ" : "BAJÓ";

  return PersonaModelo(promedio, ultimo, diferencia.abs(), estado);
}
```

2.5.4. Ejecución

The screenshot shows a mobile application interface titled "Control de Peso - Club". At the top, there are four tabs labeled "Persona 1", "Persona 2", "Persona 3", and "Persona 4", with "Persona 1" selected. Below the tabs, the section "Datos de Persona 1" contains ten input fields for weight, labeled "Peso 1" through "Peso 10". The values entered are: 67, 68, 66, 65, 68, 65, 66, 68, 69, and 67. A hamburger menu icon is visible on the left side of the input fields. Below the input fields is a button labeled "Calcular Resultados". At the bottom, a box displays the results: "Resultados: Promedio: 66.90, Último peso: 67.00, Estado: SUBIÓ, Diferencia: 0.10 kg".

Peso	Valor
Peso 1	67
Peso 2	68
Peso 3	66
Peso 4	65
Peso 5	68
Peso 6	65
Peso 7	66
Peso 8	68
Peso 9	69
Peso 10	67

Calcular Resultados

Resultados:
Promedio: 66.90
Último peso: 67.00
Estado: SUBIÓ
Diferencia: 0.10 kg

3. Conclusiones

- La abstracción de algoritmos mediante pseudocódigos y diagramas Nassi-Shneiderman ha permitido diseñar lógicas estructuradas mucho antes de escribir el código fuente, lo que previene errores semánticos y refactorizaciones durante el desarrollo.
- La adopción del patrón MVC en el entorno de desarrollo Flutter ha demostrado ser altamente eficiente para separar la lógica de negocio de la gestión de la interfaz de usuario. Extraer la lógica más importante y mostrarla de forma modular ayuda a generar un código más limpio y legible.
- En la resolución de problemas (como el cálculo de cambio o la verificación de números perfectos), encapsular la lógica pura dentro del *Modelo* permitió simplificar el código, haciendo que los controladores manejen validaciones de vista y deleguen el comportamiento matemático sin interrupciones.

4. Referencias

- Chapin, N. (1974). *New Format for Flowcharts*. Software: Practice and Experience, 4(4), 341-357.
- Google (2024). *Flutter Documentation*. Recuperado de <https://docs.flutter.dev/>
- Google (2024). *Dart Language Overview*. Recuperado de <https://dart.dev/overview>
- Google (2024). *Android Studio User Guide*. Recuperado de <https://developer.android.com/studio/intro>
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Oracle (2024). *Java Platform, Standard Edition: Java Language Specification*. Recuperado de <https://docs.oracle.com/javase/specs/>

5. Anexos

5.1. Repositorio de código fuente

5.1.1. Enlace principal

- https://github.com/LeonelTQL/moviles/tree/main/pry_tarea1